

Lineare Lernmaschinen

Artificial Intelligence: Vorlesung 8

Prof. Dr. M. O. Franz, Prof. Dr. M. Blaich

HTWG Konstanz, Fakultät für Informatik

Übersicht

- 1 Lineare Regression
- 2 Logistische Regression

Übersicht

1 Lineare Regression

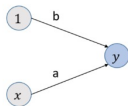
2 Logistische Regression

Wiederholung: Einfaches lineares Regressionsmodell

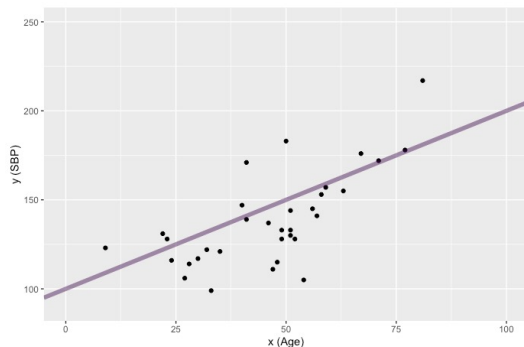
Modell:

$$\hat{y} = f_{\theta}(x) = f_{a,b}(x) = ax + b$$

mit Steigung a und y-Achsenabschnitt b .



Das Modell kann als einzelnes Neuron mit 2 Inputs und linearer Aktivierungsfunktion betrachtet werden.



$$\hat{y} = 1 \cdot x + 100$$

For age=75

$$\hat{y} = 1 \cdot 75 + 100 = 175$$

Wie findet man die besten Werte für a und b ?

Verlustfunktion MSE:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Gesucht ist eine Lösung, die den Fehler auf dem Trainingsdatensatz minimiert:

$$\hat{a}, \hat{b} = \arg \min_{a, b} \text{MSE}(a, b)$$

oder allgemein

$$\hat{\theta} = \arg \min_{\theta} \text{loss}(\theta).$$

Für die lineare Regression gibt es eine analytische Lösung (s. Vorl. 3)

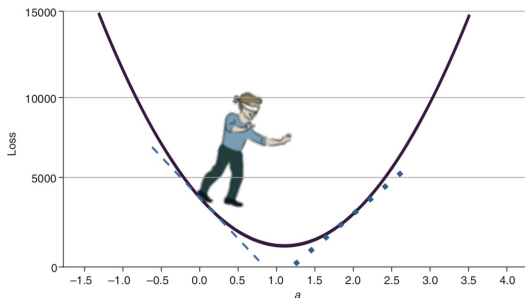
$$a = \frac{\sum_{i=1}^n (x_i - \bar{x}) \cdot (y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2} \quad \text{und} \quad b = \bar{y} - a \cdot \bar{x}.$$

Allgemein ist der *Gradientenabstieg* eine im maschinellen Lernen sehr häufig anwendbare Methode.

Eindimensionaler Gradientenabstieg

Der Einfachheit halber nehmen wir an, dass wir den optimalen Wert für $\hat{b} = 87.6$ bereits wissen:

$$\text{loss} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 = \frac{1}{N} \sum_{i=1}^N (y_i - (ax_i + 87.6))^2$$

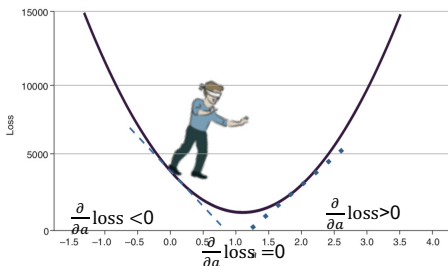


Quelle: Dürr & Sick

An jedem Punkt wissen wir den Wert der Verlustfunktion und ihren Gradienten, ohne den gesamten Verlauf zu kennen.

Gradientenabstieg: Update-Formel

Der Gradient $\frac{\partial}{\partial a}$ ist hier die Steigung a' der Verlustkurve. Ist sie positiv, so sollten wir in negative a -Richtung gehen, bei negativer Steigung in positive a -Richtung.

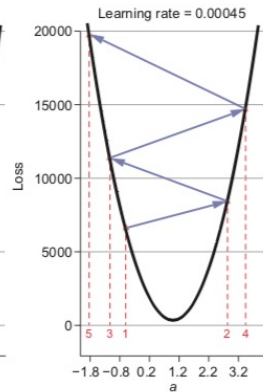
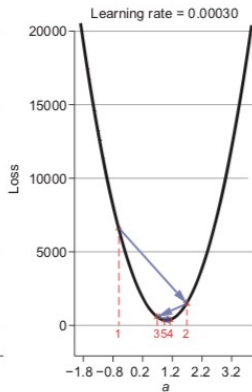
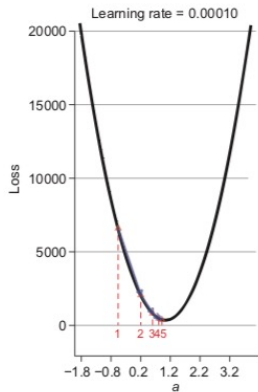


Quelle: Dürr & Sick

Update-Formel: $a_{t+1} = a_t - \eta \cdot \frac{\partial}{\partial a} \text{loss}$
 η ist die **Lernrate** bzw. Schrittweite. Hier gilt

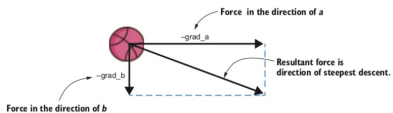
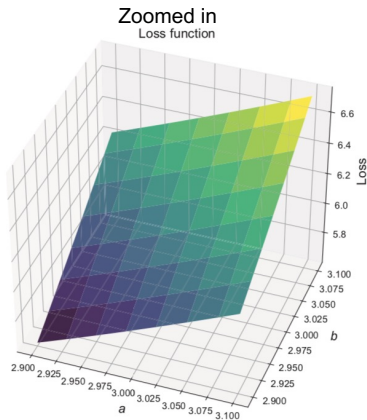
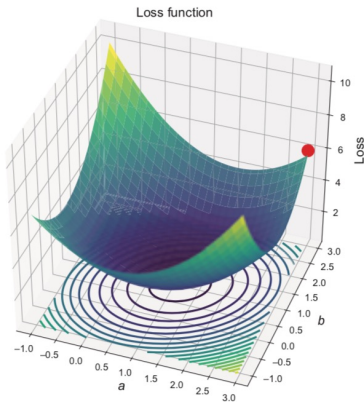
$$\begin{aligned}\frac{\partial}{\partial a} \text{loss} &= \frac{1}{N} \sum_{i=1}^N \frac{\partial}{\partial a} (y_i^2 - 2ay_ix_i - 2 \cdot 87.6y_i + a^2x_i^2 + 2 \cdot 87.6ax_i + 87.6^2) \\ &= \frac{2}{N} \sum_{i=1}^N ax_i^2 - y_ix_i + 87.6x_i = 2a \cdot \overline{x^2} - \overline{x \cdot y} + 87.6\overline{x}\end{aligned}$$

Wirkung der Lernrate η



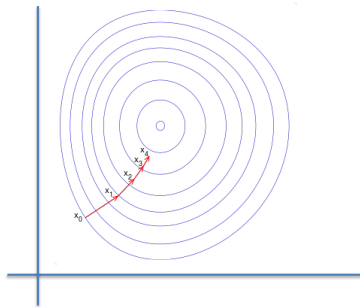
Quelle: Dürr & Sick

Zweidimensionale Verlustfunktion



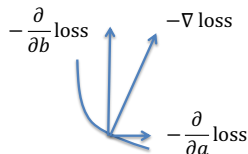
Quelle: Dürr & Sick

Zweidimensionaler Gradientenabstieg



Quelle: Dürr & Sick

Der Gradientenabstieg erfolgt senkrecht zu den Höhenlinien.



$$a_{t+1} = a_t - \eta \cdot \frac{\partial}{\partial a} \text{loss}$$

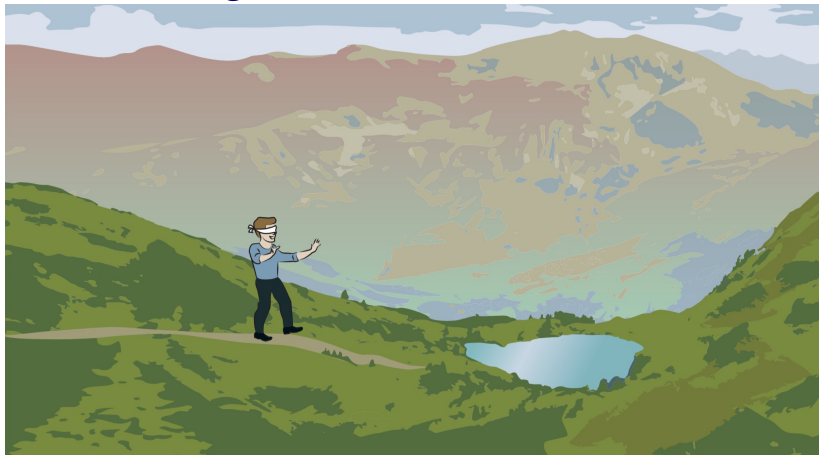
$$b_{t+1} = b_t - \eta \cdot \frac{\partial}{\partial b} \text{loss}$$

$$\begin{pmatrix} a_{t+1} \\ b_{t+1} \end{pmatrix} = \begin{pmatrix} a_t \\ b_t \end{pmatrix} - \eta \nabla \text{loss}$$

Allgemein:

$$\theta_{t+1} = \theta_t - \eta \nabla \text{loss}$$

Gradientenabstieg auf nichtkonvexer Verlustfunktion



Quelle: Dürr & Sick

Bei bestimmten Optimierungsproblemen (konvexe Probleme, z.B. lineare Regression) sind lokale und globale Maxima identisch, bei anderen nicht (z.B. neuronale Netze). Der Gradientenabstieg findet immer nur **lokale Minima**.

Multivariate lineare Regression

	TV	Radio	Newspaper	Sales
0	230.1	37.8	69.2	22.1
1	44.5	39.3	45.1	10.4
2	17.2	45.9	69.3	9.3
3	151.5	41.3	58.5	18.5
4	180.8	10.8	58.4	12.9
...	...	...	...	...
195	38.2	3.7	13.8	7.6
196	94.2	4.9	8.1	9.7
197	177.0	9.3	6.4	12.8
198	283.6	42.0	66.2	25.5
199	232.1	8.6	8.7	13.4

200 rows x 4 columns

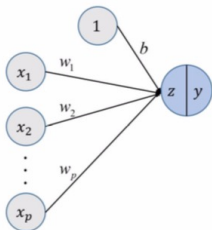
Quelle: Dürr & Sick

Lineares multivariates Modell:

$$\hat{y} = w_0 \cdot 1 + w_1 x_1 + w_2 x_2 + \dots + w_d x_d$$

Der Schwellwert (Bias) w_0 wird oft als b geschrieben.

Interpretation als abstraktes Neuron mit linearer Aktivierungsfunktion:

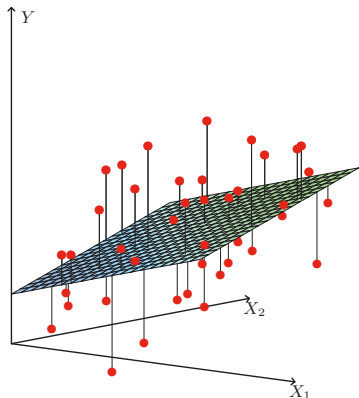


Quelle: Dürr & Sick

$$\widehat{\text{sales}} = 1 \cdot w_0 + w_1 \text{TV} + w_2 \text{Radio} + w_3 \text{Newspaper}$$

Geometrische Interpretation: Fit einer Hyperebene

Matrixschreibweise:



Quelle: Hastie et al., 2009

Intercept	TV	Radio	Newspaper
1.0	230.1	37.8	69.2
1.0	44.5	39.3	45.1
1.0	17.2	45.9	69.3
1.0	151.5	41.3	58.5
1.0	180.8	10.8	58.4
...	...	...	...
1.0	38.2	3.7	13.8
1.0	94.2	4.9	8.1
1.0	177.0	9.3	6.4
1.0	283.6	42.0	66.2
1.0	232.1	8.6	8.7

$$\begin{pmatrix} w_0 \\ w_1 \\ w_2 \\ w_3 \end{pmatrix}$$

y.hat
20.244783
12.426815
12.310875
17.457258
13.204685
...
5.815722
8.534241
13.000720
23.386213
15.297458

Sales
22.1
10.4
9.3
18.5
12.9
...
7.6
9.7
12.8
25.5
13.4

Quelle: Dürr & Sick

mit der **Designmatrix** X

$$X\mathbf{w} = \hat{\mathbf{y}}$$

und der Verlustfunktion

$$\text{MSE}(\mathbf{w}) = \frac{1}{N} \|\mathbf{y} - \hat{\mathbf{y}}\|^2 = \|\mathbf{y} - X\mathbf{w}\|^2$$

Das Optimierungsproblem ist hier vierdimensional in den Variablen w_0, w_1, w_2, w_3 . Lösung entweder analytisch oder über Gradientenabstieg.

Übersicht

1 Lineare Regression

2 Logistische Regression

Logistische Regression (Motivation)

- Kreditscoring & logistische Regression



67,7 Mio. Personen und 6 Mio. Unternehmen

Anzahl und Art der Kreditaktivitäten

Zahlungsausfälle

Erfahrungen im Umgang mit Kreditgeschäften

„Das verwendete Verfahren wird als „logistische Regression“ bezeichnet und ist eine fundierte, seit langem praxiserprobte, mathematisch-statistische Methode zur Prognose von Risikowahrscheinlichkeiten.“

Quelle: Constanze Riedinger, Data Analytics

Logistische Regression

- Erweitert die Idee der linearen Regression auf Situationen, in denen die Zielvariable kategorial ist
- Wir konzentrieren uns auf binäre Klassifikation
d. h. $Y = 0$ oder $Y = 1$
- Ist eine probabilistische Methode und liefert die Wahrscheinlichkeit für Klasse $Y = 1$

Logistische Regression(Motivation Fortsetzung)

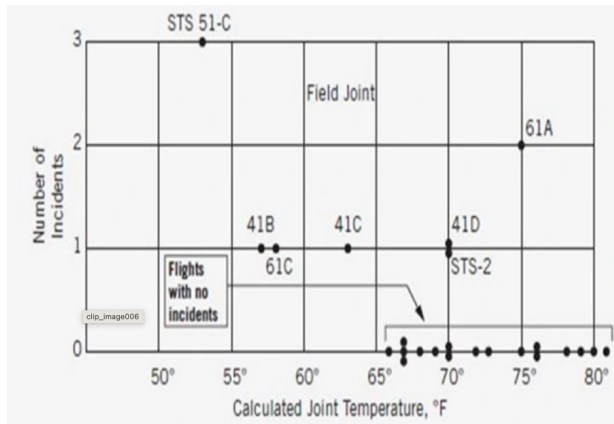
Hintergrund: probabilistische Modellierung



Die Challenger-Raumfähre explodierte 73 Sekunden nach dem Start im Jahr 1986. Eine Dichtung im Booster war defekt.

Statistik & Challenger-Desaster [Exkurs]

Auch in den erfolgreichen Flügen steckt Information.

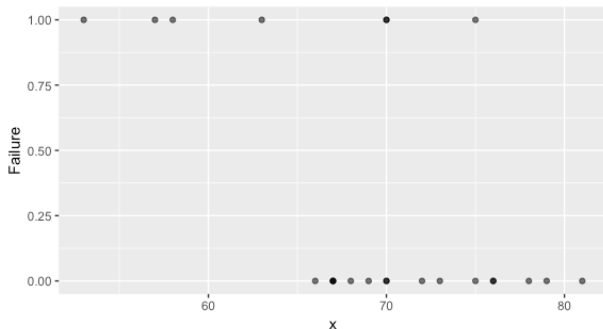


Modellierung mit logistischer Regression

Binarisierung: Null-/Eins-Klassifikation

Gesucht: $p(x) = \Pr(Y = 1 \mid x)$

Wahrscheinlichkeit für einen O-Ring-Defekt $Y = 1$ bei gegebener Temperatur X



- Welche Form hat die Kurve?
- Warum ist $p(X) = b + ax$ (lineare Regression) **ungeeignet**?

Suche nach einer geeigneten Kompressionsfunktion

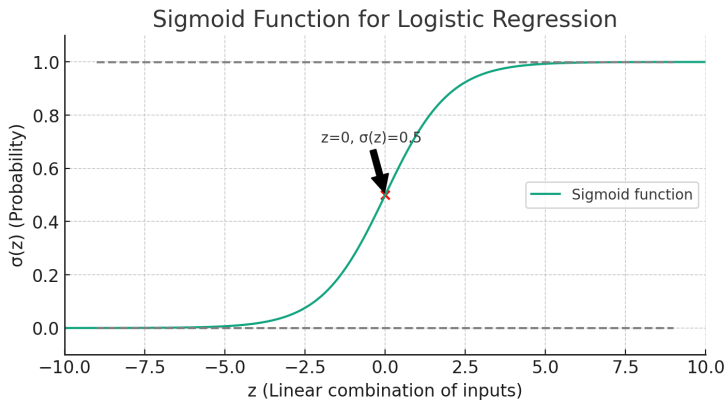
- Idee der logistischen Regression:
 - Ausgabe der linearen Regression verwenden:

$$z = a \cdot x + b, \quad z \in [-\infty, \infty]$$

- und auf $[0, 1]$ komprimieren.
- **Aufgabe: Skizzieren Sie eine Funktion, die das leisten könnte.**
 - Diskutieren Sie mit Ihrem Nachbarn

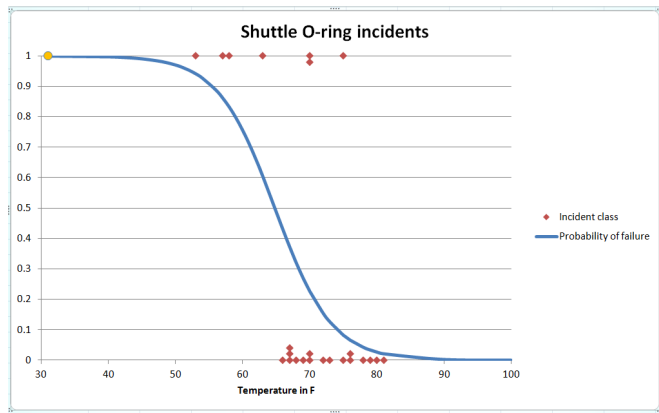
Lösung

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



Logistische Regression

Vorhersage: Ist der O-Ring defekt, in Abhängigkeit von der Temperatur?



Statistisches Modell (logistische Regression):

$$P(Y = 1 \mid X = x) = (1 + e^{-(ax+b)})^{-1} = (1 + e^{-z})^{-1} = f(x)$$

$$z \in] - \infty, \infty[\rightarrow [0, 1]$$

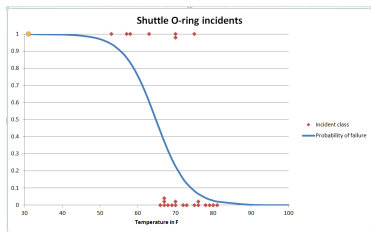
Maximum-Likelihood-Prinzip

- Lineare Regression minimiert den RMSE
- Der RMSE der linearen Regression lässt sich als Maximum-Likelihood-Methode auffassen
- Dies ist eine allgemeine Verlustfunktion

$$\text{loss} = -\frac{1}{N} \sum_{i=1}^N \log(p_{\text{model}}(y^{(i)} \mid x^{(i)}; \boldsymbol{\theta}))$$

Wahrscheinlichkeit, die das Modell dem beobachteten Ergebnis y_i zuweist

Likelihood: Wahrscheinlichkeit des Trainingssets



Trainingsdaten $i = 1, \dots, N$: $X^{(i)}, Y^{(i)}$

$$p_1(X) = P(Y = 1 \mid X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x)}}$$

Wahrscheinlichkeit für $Y = 1$ bei gegebenem X

$$p_0(X) = 1 - p_1(X)$$

Wahrscheinlichkeit für $Y = 0$ bei gegebenem X

Likelihood (Wahrscheinlichkeit des Trainingssets bei gegebenen Parametern):

$$L(a, b) = \prod_{i \in \text{alle Einsen}} p_1(x^{(i)}) \cdot \prod_{j \in \text{alle Nullen}} p_0(x^{(j)})$$

⇒ Diese Wahrscheinlichkeit soll maximiert werden

Maximierung der Likelihood

Likelihood bzgl. der Parameter maximieren:

$$L(a, b) = \prod_{i \in \text{alle Einsen}} p_1(x^{(i)}) \cdot \prod_{j \in \text{alle Nullen}} p_0(x^{(j)})$$

Logarithmus anwenden (Maximum des Logarithmus liegt an derselben Stelle):

$$\begin{aligned} -N\mathcal{J}(\theta) &= L(\theta) = L(a, b) \\ &= \sum_{i \in \text{alle Einsen}} \log(p_1(x^{(i)})) + \sum_{i \in \text{alle Nullen}} \log(p_0(x^{(i)})) \\ &= \sum_{i \in \text{alle Daten}} y_i \log(p_1(x^{(i)})) + (1 - y_i) \log(p_0(x^{(i)})) \end{aligned}$$

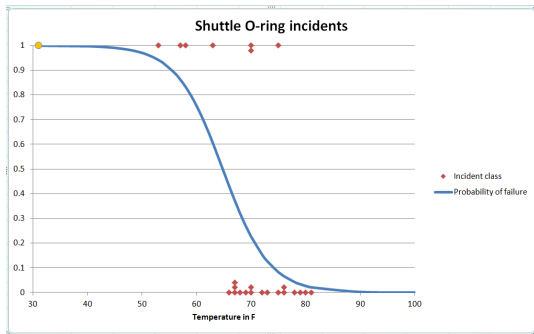
Verlustfunktion (negative Log-Likelihood, Kreuzentropie):

$$\text{loss} = -\frac{1}{N} \sum_{i=1}^N \log(p_{\text{model}}(y^{(i)} \mid x^{(i)}; \theta))$$

Anpassen der Parameter

Ergebnis:

$$\text{loss} = -\frac{1}{N} \sum_{i=1}^N \log(p_{\text{model}}(y^{(i)} | x^{(i)}; \theta))$$



Diese Kurve hängt von den Parametern a , b ab.

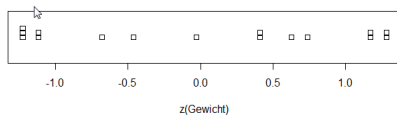
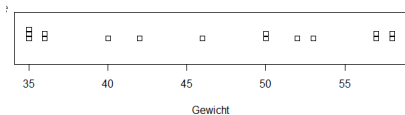
Wähle a und b so, dass die Daten optimal angepasst werden.

Skalierung

- Die logistische Regression (und jedes neuronale Netz) hängt von der Skalierung von X ab.
- Es ist ungünstig, wenn verschiedene Spalten von X unterschiedliche Wertebereiche haben
- Trick (z-Transformation, auch: Standardskalierung): Mittelwert subtrahieren und durch Standardabweichung dividieren:

$$x \mapsto z(x) = \frac{x - \bar{x}}{sd_x} = \frac{1}{sd_x} x - \frac{\bar{x}}{sd_x}$$

- Beispiel (Klopapier):



Skalierung in Scikit-Learn

```
In [91]: import numpy as np
import sklearn
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
X, y = load_iris(return_X_y=True)
print(X[0:4,])

[[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]]
```

```
In [92]: # Scaling
# Learning the scaler
scaler = sklearn.preprocessing.StandardScaler().fit(X)
X = scaler.transform(X)
print(X[0:4,])
np.mean(X, axis=0) #Close to zero
np.std(X, axis=0)  #Close to one

[[-0.90068117  1.01900435 -1.34022653 -1.3154443 ]
 [-1.14301691 -0.13197948 -1.34022653 -1.3154443 ]
 [-1.38535265  0.32841405 -1.39706395 -1.3154443 ]
 [-1.50652052  0.09821729 -1.2833891  -1.3154443 ]]
```

```
Out[92]: array([1., 1., 1., 1.])
```

Bei Training- und Testset: Skalierer **nur** auf dem Trainingsset anpassen.